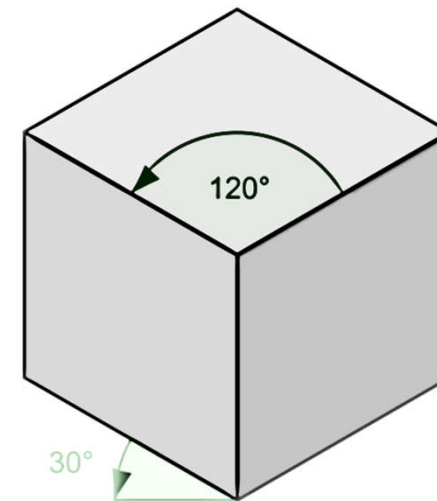
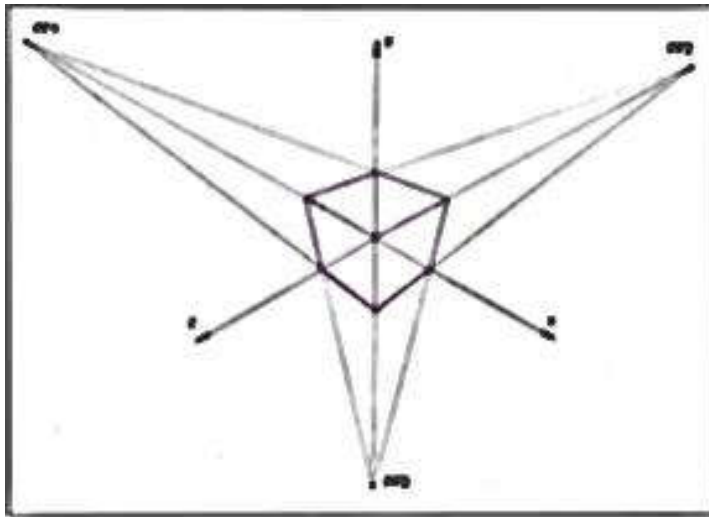
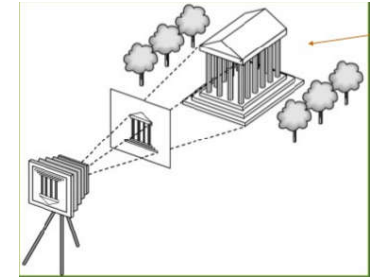


Visión 2D, 3D



Temario

Tema 1: Introducción: Revisión histórica da computación gráfica

Tema 2: Estándares gráficos Evolución histórica, OpenGL vs DirectX

Tema 3: Formas 2D e Antialiasing (final del curso)

Tema 4: Transformacións xeométricas, 2D, 3D,

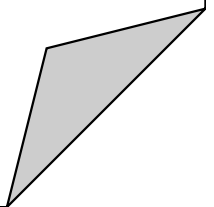
Tema 5: Proyeccións, modelo de cámara sintética.

Tema 6: Modelado e texturas

Tema 7: Color, Iluminación y sombreado

Tema 8: Determinación de superficies visibles, z-Buffer

Tema 9: Shaders GLSL



Tema 5: Proyecciones, modelo de cámara sintética

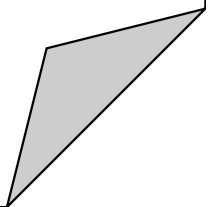
Tema 5: Proyecciones, modelo de cámara sintética

1. Taxonomía de las proyecciones.

1.1. Paralelas.

1.2. Perspectiva.

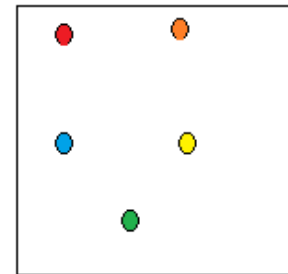
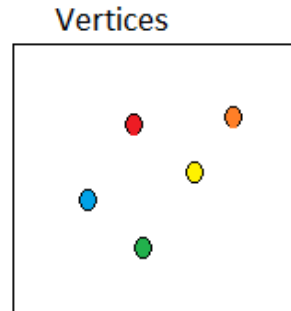
2. Modelo de Cámara sintética.



Pipe 3.3

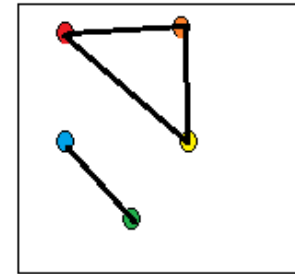
Vertex
Shader

Aplicación

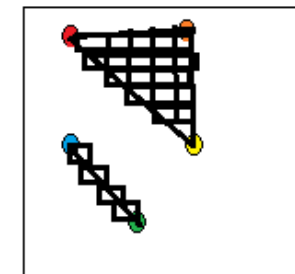


Assembly

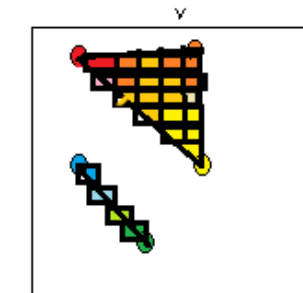
Ensamblado



Raster



Interpola

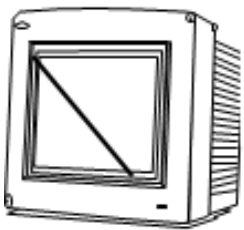


Colorear los fragmentos

Fragmentos

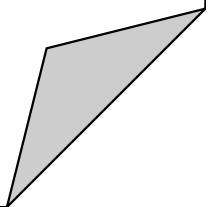
Fragment
shader

Dibujo Color
fragmento
s z-buffer

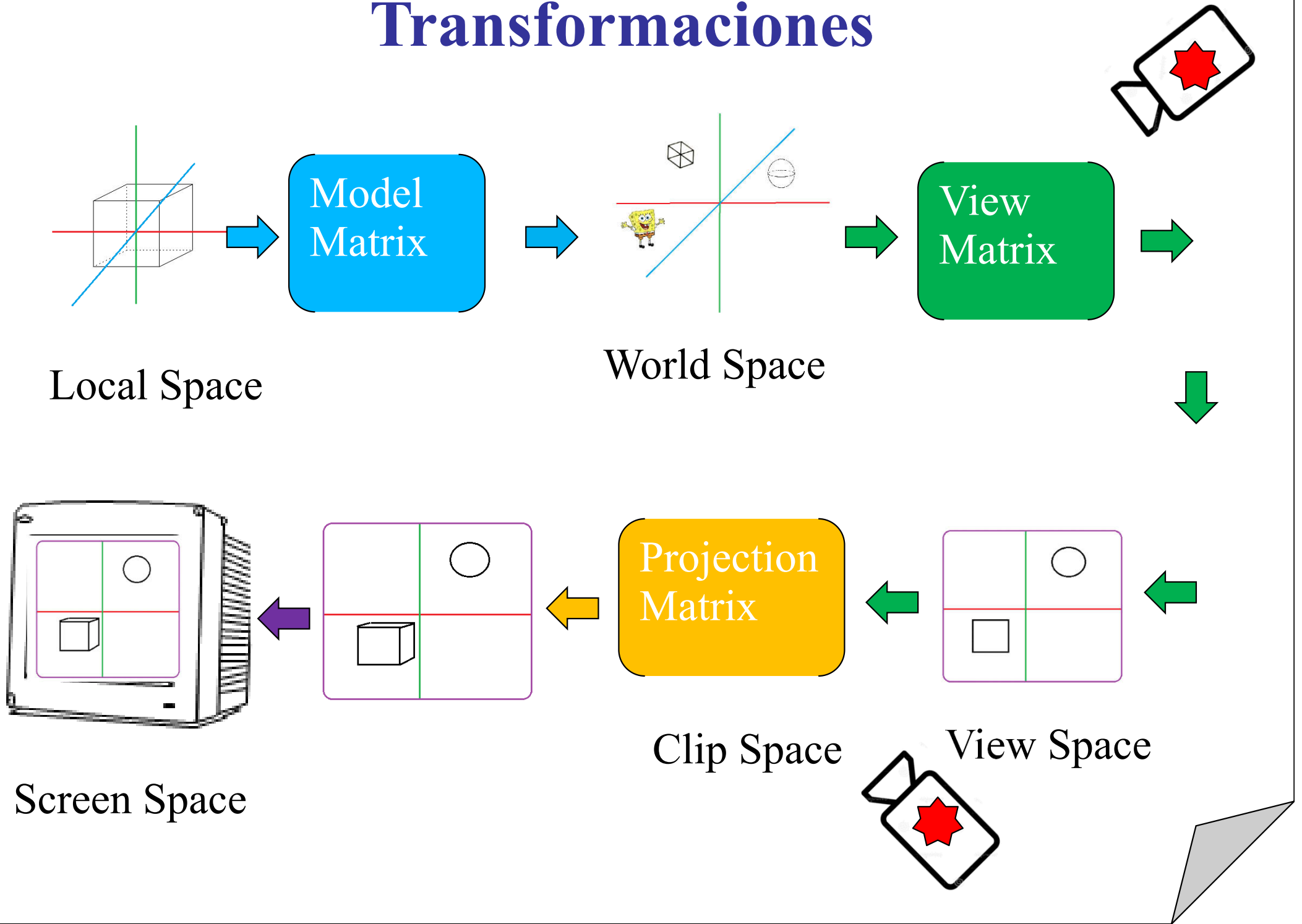


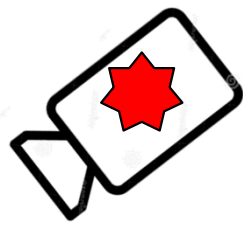
Shaders

- **Vertex shaders.**
 - Función que recibe como parámetro vértices.
 - Manipula vértices en 3D; Color, coordenadas de textura, orientación y tamaño de vértices.
 - Su propósito es transformar las coordenadas 3D de los vértices en otras coordenadas del dispositivo.
 - La salida debe ser la posición de los vértices.
- **Fragment shaders (OpenGL)**
 - Recibe la información de cada fragmento.
 - Un fragmento es toda la información necesaria para dibujar un pixel.
 - Se aplica en la última etapa de rasterizado.
 - Su propósito es obtener el color de cada pixel.
 - La salida es el color de cada pixel.
- **Que pasa después del Fragment Shader**
 - Se realizan una serie de comprobaciones relativas al Alpha test, blending test y Depth test.

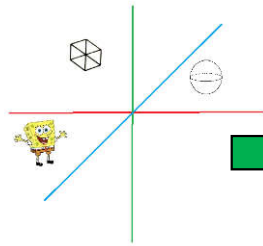
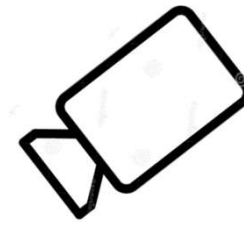


Transformaciones

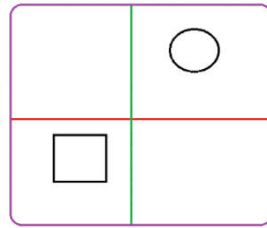




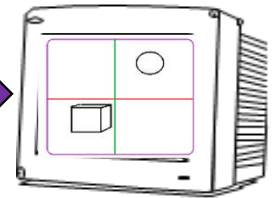
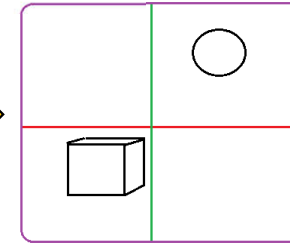
View Space



View
Matrix



Projection
Matrix



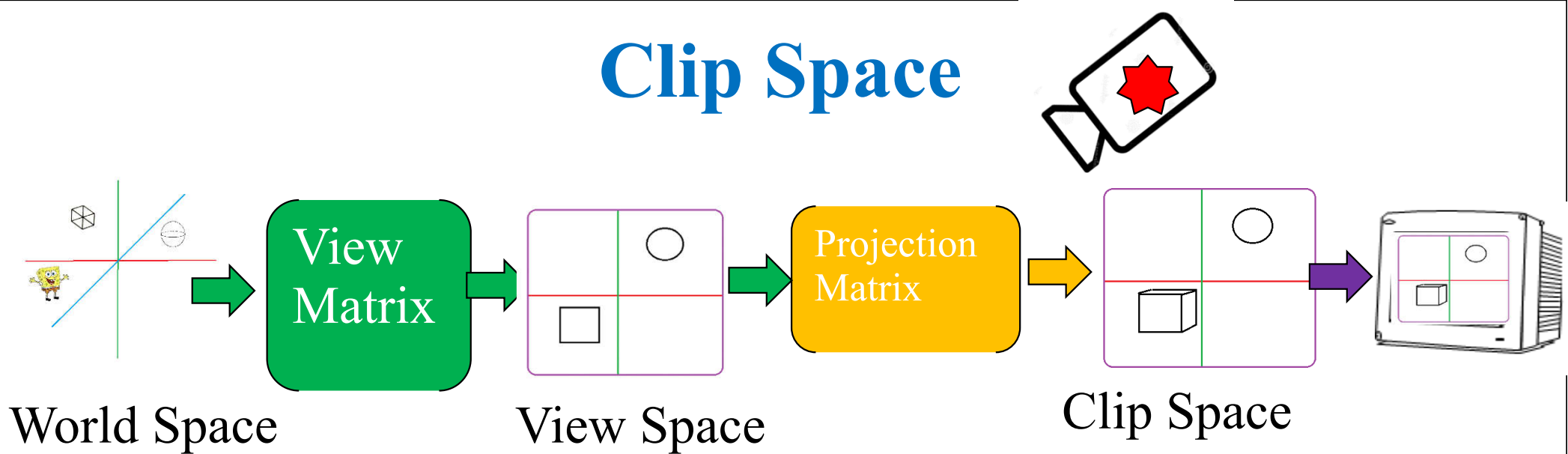
World Space

View Space

Clip Space

- El **View Space (Espacio de Vista)** es el sistema de coordenadas asociado a la cámara o **punto de vista del observador**. Se obtiene transformando las coordenadas del World Space al sistema de coordenadas de la cámara mediante una combinación de traslaciones y rotaciones.
- Esta transformación se realiza aplicando la matriz de vista (**View Matrix**) a las coordenadas de la escena.
- En el espacio de vista, la **cámara por defecto** siempre se ubica en el origen **(0, 0, 0)** y **mira hacia el eje -Z**. Esto significa que los objetos que aparecerán en la imagen final son aquellos que se encuentran frente a la cámara en este espacio, pero podemos moverla a donde consideremos oportuno.

Clip Space

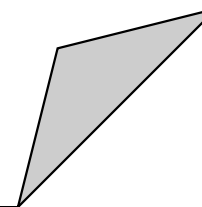


- Clip Space:** Una vez obtenidas las coordenadas en el espacio de vista (**view space**), se realiza un **proceso de recorte y proyección (clipping)** para descartar los elementos que se encuentran fuera del volumen definido por el *frustum*. Los vértices que quedan fuera de este volumen son eliminados, mientras que los restantes se procesan para su posterior conversión **en fragmentos**.

- Por defecto, para ser visibles, las coordenadas finales deben estar dentro del rango de **-1.0 a 1.0** en todos los ejes. Este rango corresponde al espacio de coordenadas normalizadas (**Normalized Device Coordinates, NDC**). Este volumen se configura mediante la **matriz de proyección**, la cual define el espacio visible y escala las coordenadas a dicho rango.

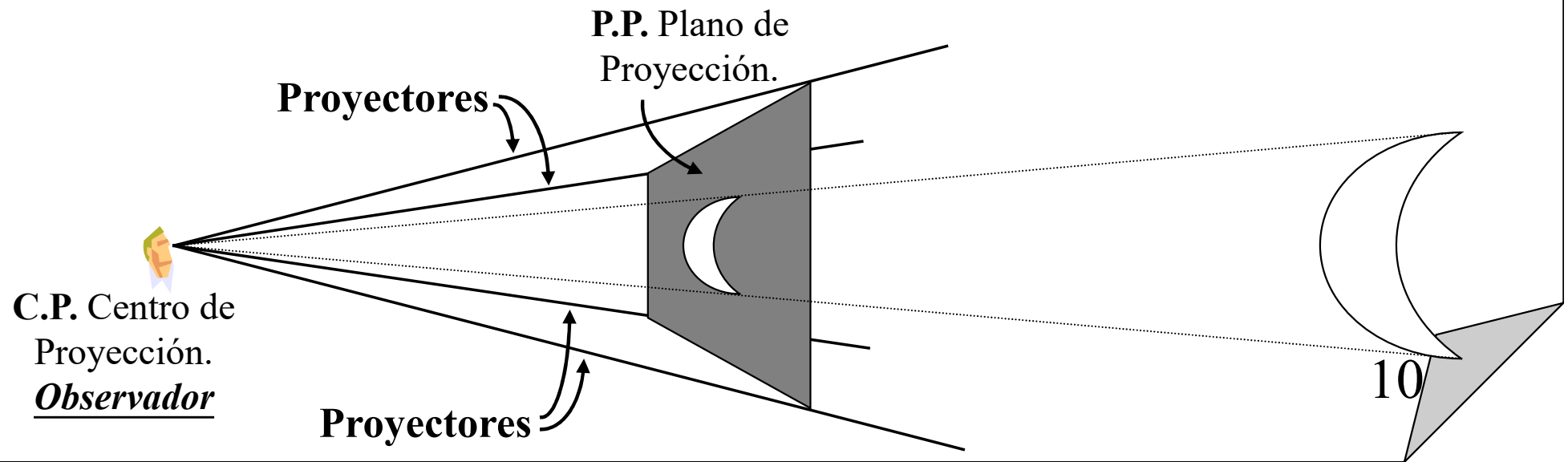
- Frustum** se termina mediante una proyección

Tutorial Projection



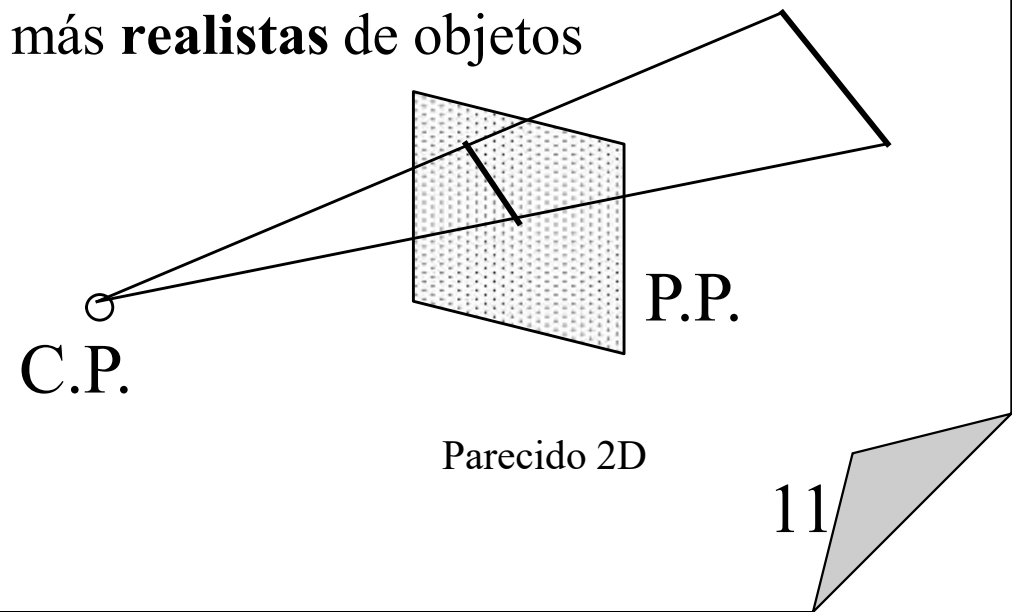
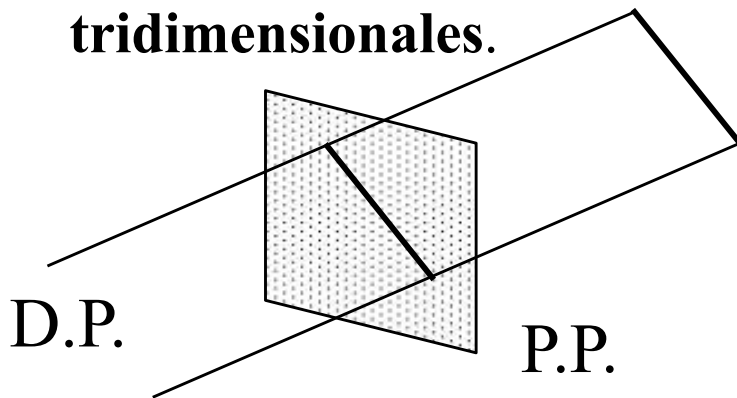
Proyección

- Una proyección es una **transformación** que **reduce** la dimensionalidad de un espacio, típicamente de $\mathbf{R}^3$ a $\mathbf{R}^2$, proceso de convertir una escena tridimensional en una imagen bidimensional. Elementos de una Proyección:
- **Objetos a proyectar (escena):** Los elementos tridimensionales que componen el mundo virtual.
- **Plano de proyección:** La superficie 2D donde se representan los objetos 3D. En términos prácticos, esto corresponde a **la pantalla o viewport**.
- **Punto de vista o centro de proyección:** La posición desde donde se observa la escena. Representa la ubicación de la cámara virtual.
- **Proyectores:** Rayos imaginarios que se trazan desde los puntos del objeto hasta el centro de proyección. Estos rayos intersecan el plano de proyección, determinando dónde se dibujará cada punto en la imagen final.

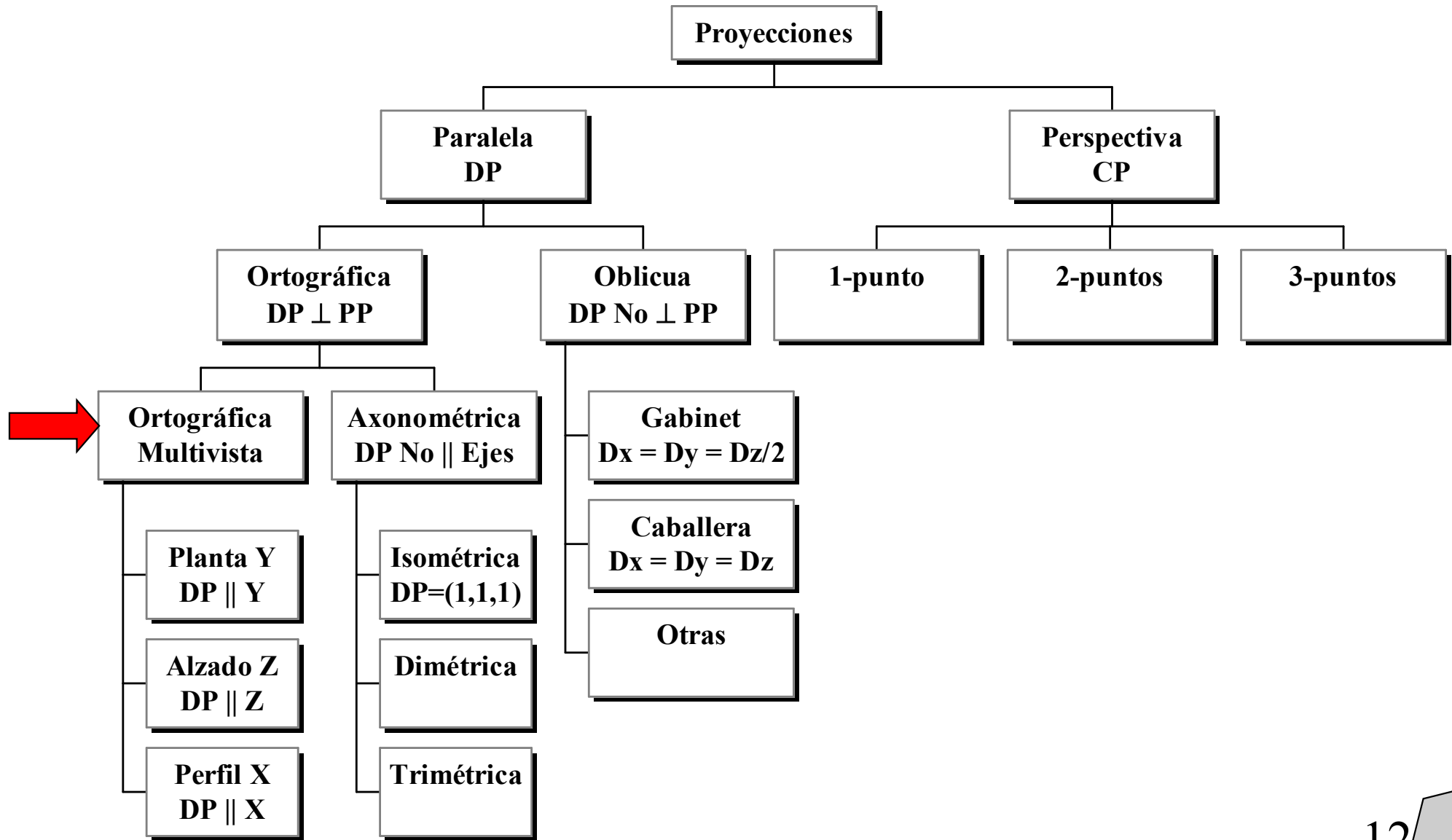


Tipos de proyecciones

- Proyección Paralela (**ortográfica**), se caracteriza por:
 - El **centro de proyección** está situado en el **infinito**.
 - Los **proyectores** son **paralelos** entre sí.
 - Se define una **dirección de proyección (D.P.)** en lugar de un centro de proyección.
- La proyección **perspectiva**, se caracteriza por:
 - **Existe un centro de proyección (C.P.)** en el espacio.
 - Los rayos proyectantes forman un **cono** cuyo vértice es el **punto de proyección**.
 - Se utiliza para crear representaciones más **realistas** de objetos **tridimensionales**.



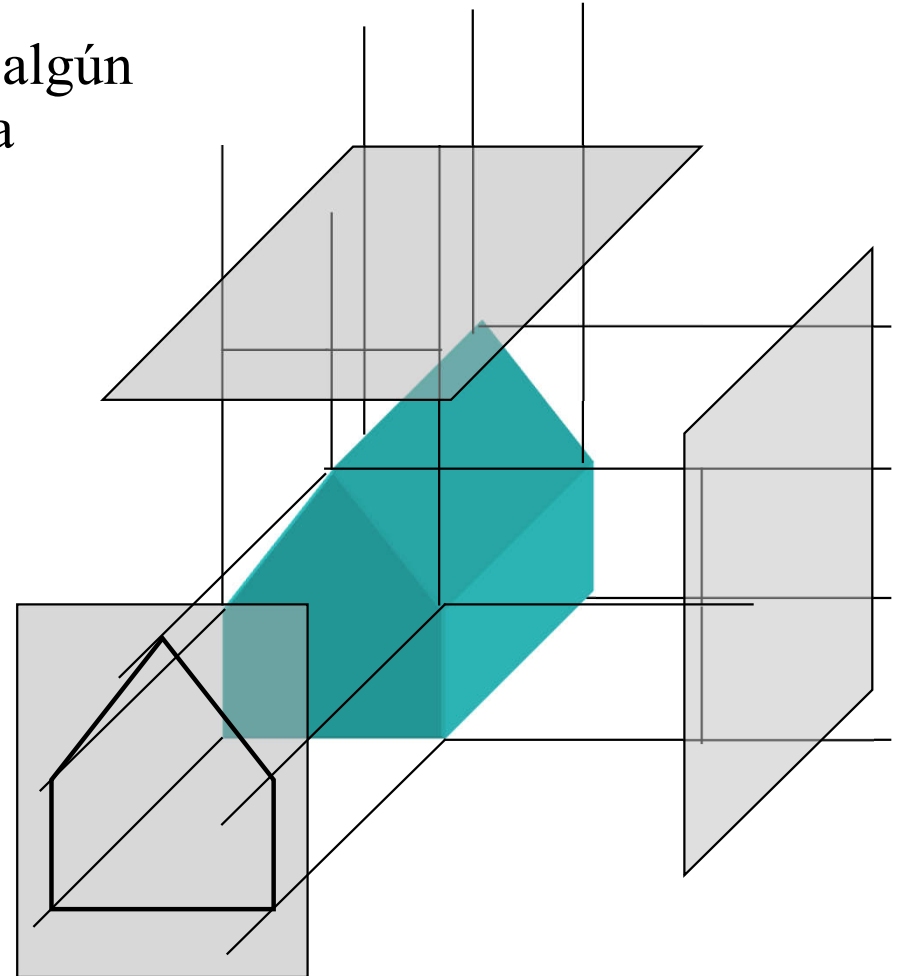
Clasificación Proyecciones



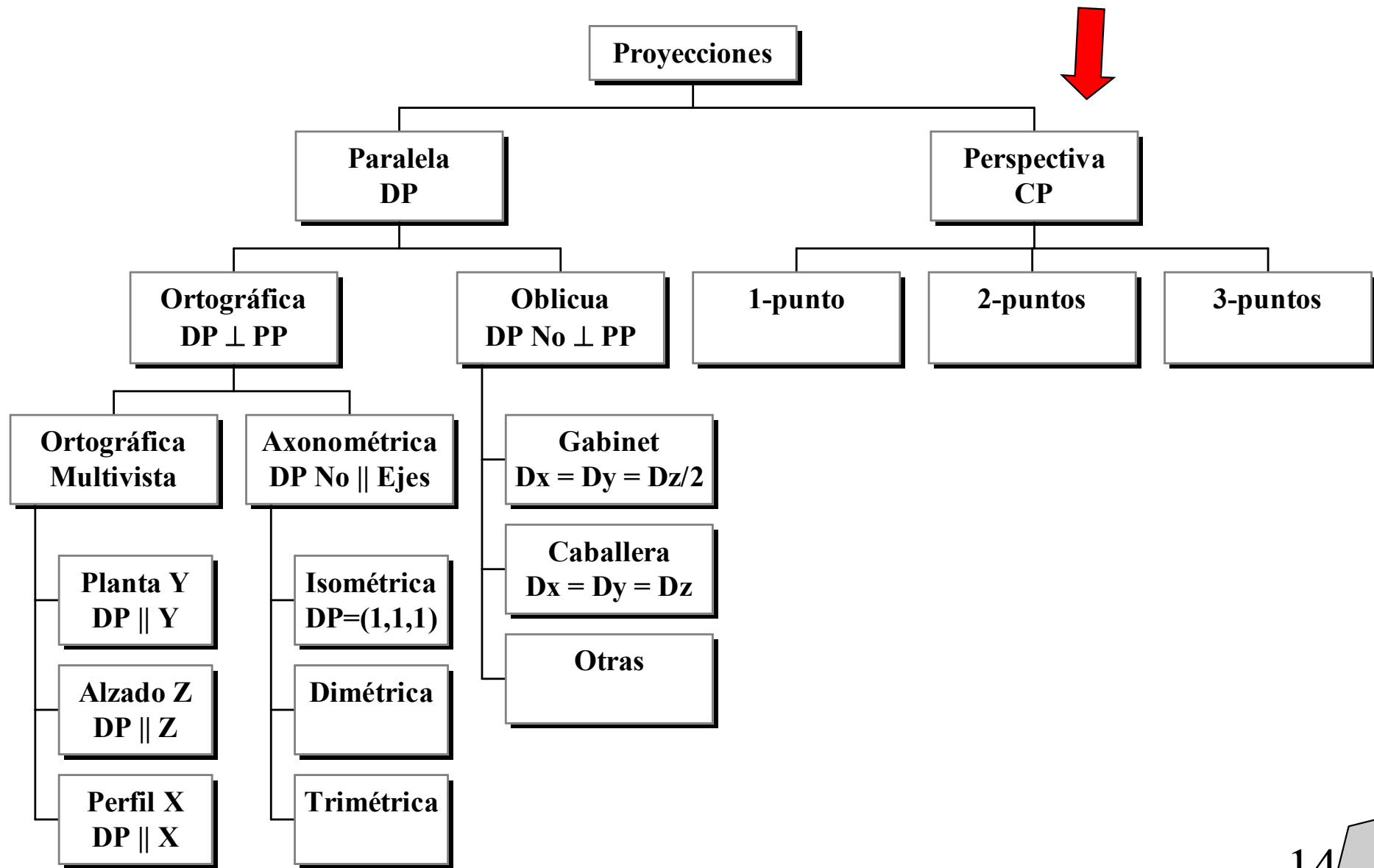
Clasificación Proyecciones: Multivista

Ortográfica $DP \perp PP$

- El **plano de proyección** es **perpendicular** a algún **eje**. Necesitamos **varias vistas** para tener una **visión 3D** del objeto.
 - Planta: $PP \perp$ eje Y.
 - Alzado: $PP \perp$ eje Z.
 - Perfil: $PP \perp$ eje X.
- Ventajas.
 - Es posible realizar **medidas precisas**.
 - Todas las vistas tienen la **misma escala**.
- Inconvenientes.
 - No obtenemos una visión 3D del objeto.
- Se utilizan en:
 - Diseños de **ingeniería**.
 - Planos de **arquitectura**.

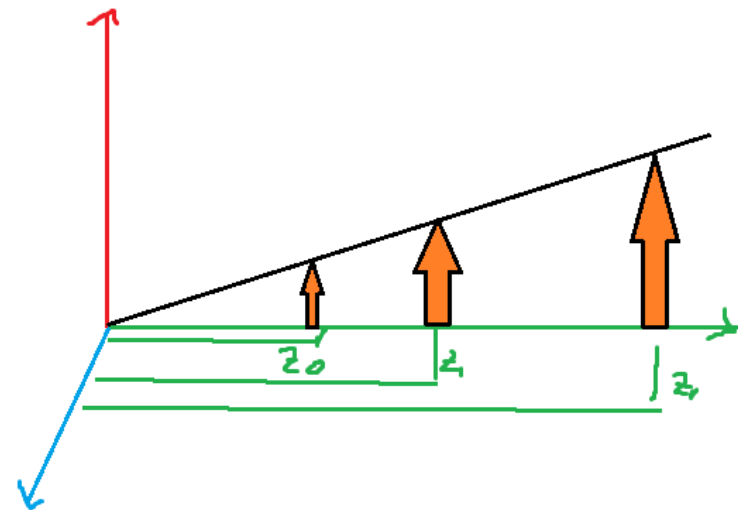
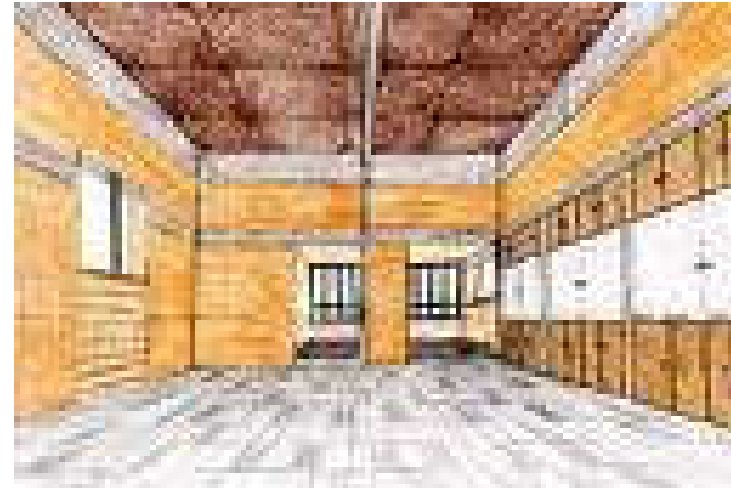


Clasificación Proyecciones

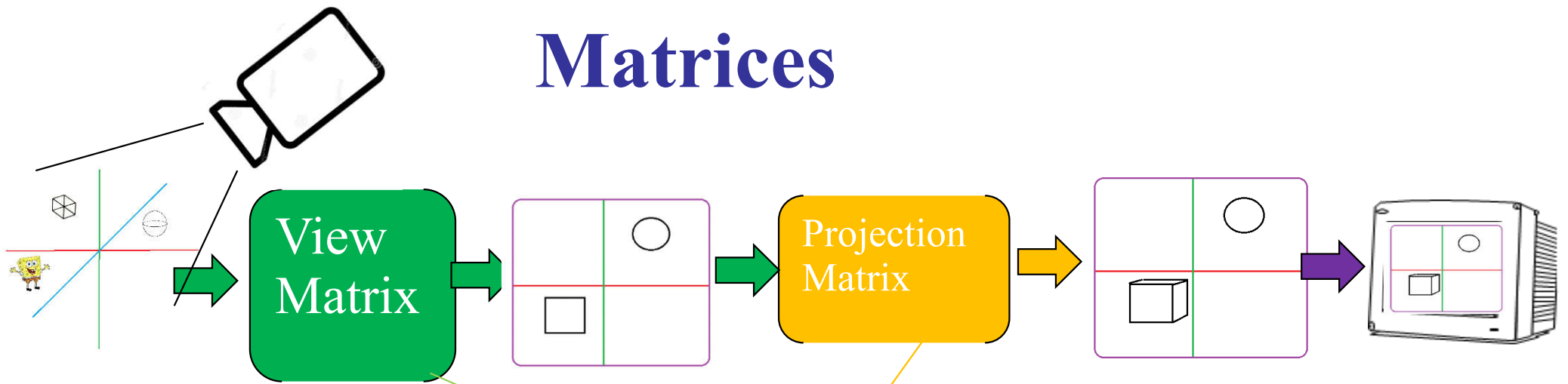


Proyecciones: Perspectiva

- Ventajas.
 - Proporcionan **realismo visual**.
 - Sensación **tridimensional**.
- Inconvenientes.
 - **No mantienen la forma**.
 - **No mantienen la escala**.
- Uso.
 - **Publicidad, arte, animación, resultados finales arquitectónicos, diseño de interiores...**



Matrices



En OpenGL 3.3 debemos de crea una matriz para cada uno de los pasos y componer una ecuación tal como:

$$Vclipposition = Mprojection * Mview * Mmodel * Vertices$$

Vértices.- las coordenadas de los vértices.

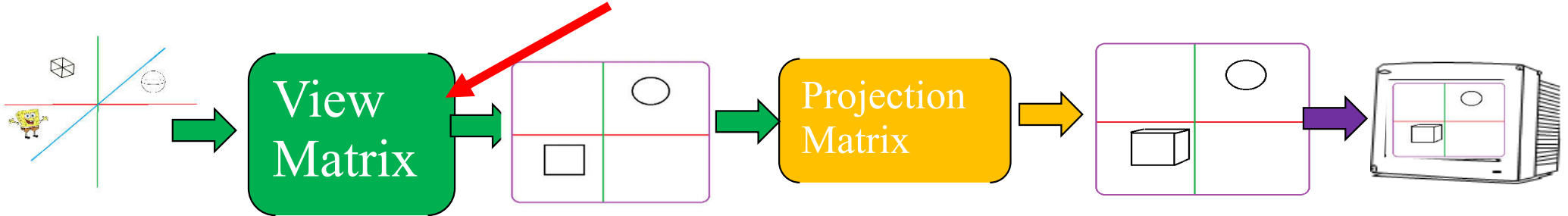
Mmodel.- la matriz de transformación.

Mview.- la matriz de vista (donde está la cámara)

Mproyection.- La matriz de perspectiva.

La salida *Vclipposition* debe ser aplicada *gl_Position* por tanto en el *vertex.shader*.

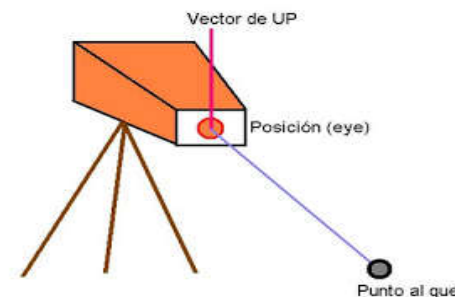
ViewMatrix (a donde mira la cámara)



- Determina dónde y cómo está dispuesta la cámara. No tiene nada que ver con el tipo de proyección. Es literalmente la posición XYZ dónde colocar la cámara dentro del mundo. Coordenadas del "at". Es el valor XYZ del punto al que queremos que mire la cámara. Coordenadas del vector "up". Si, es un vector y no un punto. Con él regularemos la orientación de la cámara.
- `gluLookAt( posicion.x, posicion.y, posicion.z, direccion.x, direccion.y, direccion.z, 0,1,0);`
- Por defecto la cámara en el 0,0,0, dirección 0,0,-1 orientación 0,1,0. Para generar los diferentes planos bastará con girar la cámara.

Mview. la calculamos con la matriz del modelo

```
glm::mat4 view (1.0);  
view = glm::lookAt(glm::vec3(x,y,z), glm::vec3(x,y,z), glm::vec3(x,y,z));
```



*M*projection ortográfica

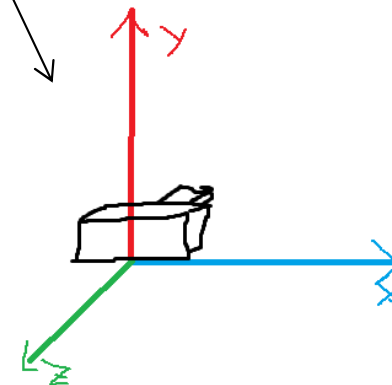
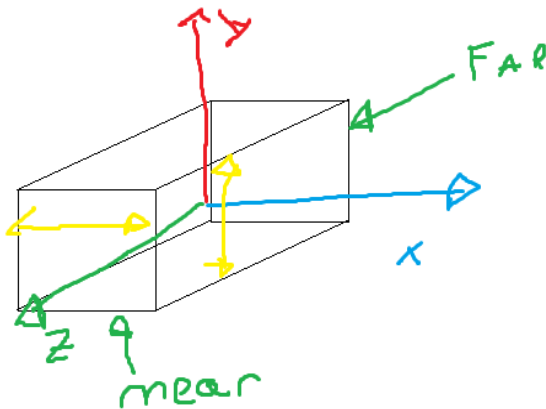
Proyección ortográfica.

- Volumen ortográfico

```
glm::mat4 proj = glm::ortho(-100.0f, 100.0f, -100.0f, 100.0f, 0.1f, 100.0f);
```

Por defecto los valores son `glOrtho3D(-1, 1, 1, -1, -1, 1)`; Para generar las diferentes **perspectivas multivista** bastará con girar la cámara.

*M*projection



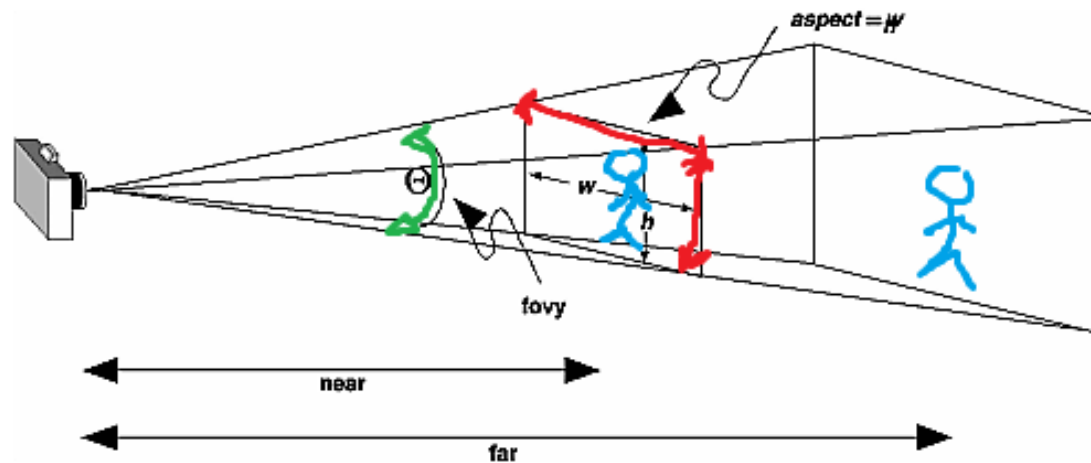
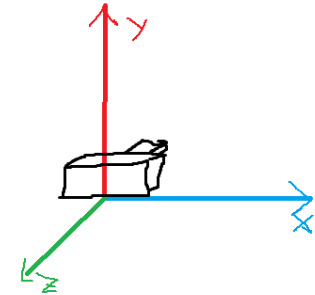
glOrtho describes a transformation that produces a parallel projection. The current matrix (see `glMatrixMode`) is multiplied by this matrix and the result replaces the current matrix, as if `glMultMatrix` were called with the following matrix as its argument:

Perspective projection

- Volumen en perspectiva

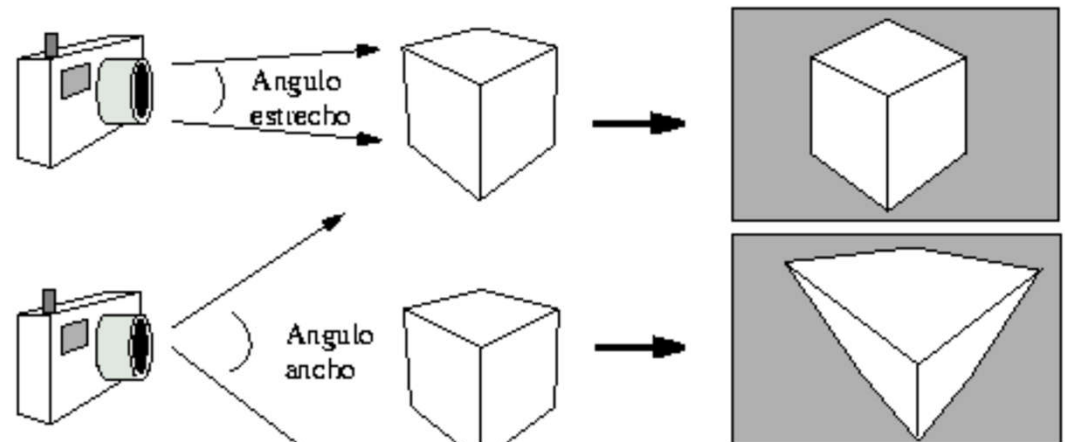
```
glm::mat4 proj = glm::perspective(45.0f, Aspect, 0.1f, 100.0f);
```

Aspect = (**float**)width/(**float**)height



Modelo De Cámara Sintética

- Razón de aspecto
 - Relación entre el ancho y alto
 - Relaciones típicas son 1:1, 4:3, 16:9, depende de tu tamaño de ventana.
- FOV
 - Va desde 40 a 179 grados. Es equivalente a introducir lentes de campo en la cámara.
 - Ej: Ojo de pez (Mirilla)



Mview-Mproyection-GLM

Mview. la calculamos con la matriz del modelo

```
glm::mat4 view;  
view = glm::lookAt(glm::vec3(x,y,z), glm::vec3(x,y,z), glm::vec3(x,y,z));
```

Mprojection

- Volumen ortográfico

```
glm::mat4 proj = glm::ortho(-100.0f, 100.0f, -100.0f, 100.0f, 0.1f, 100.0f);
```

- Volumen en perspectiva

```
glm::mat4 proj = glm::perspective(45.0f, Aspect, 0.1f, 100.0f);
```

Aspect = (float)width/(float)height

Como es el shader

$$Vclip = Mprojection * Mview * Mmodel * Vertices$$

```
glm::mat4 model; // Se calcula
glm::mat4 view; // Se Calcula
glm::mat4 projection; // Se calcula

unsigned int modelLoc = glGetUniformLocation(IDShader, "model");
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));

unsigned int viewLoc = glGetUniformLocation(IDShader, "view");
glm::mat4 view = glm::lookAt(glm::vec3(x,y,z), glm::vec3(x,y,z), glm::vec3(x,y,z));
glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));

unsigned int projectionLoc = glGetUniformLocation(IDShader, "projection");
glm::mat4 projection = glm::ortho(-100.0f, 100.0f, -100.0f, 100.0f, 0.1f, 100.0f);
glUniformMatrix4fv(projectionLoc, 1, GL_FALSE, glm::value_ptr(projection));
```

```
#version 330 core
layout (location = 0) in vec3 aPos;

uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

void main() {
gl_Position = projection * view * model * vec4(aPos, 1.0);
}
```

Matriz Proyeccion ortográfica.

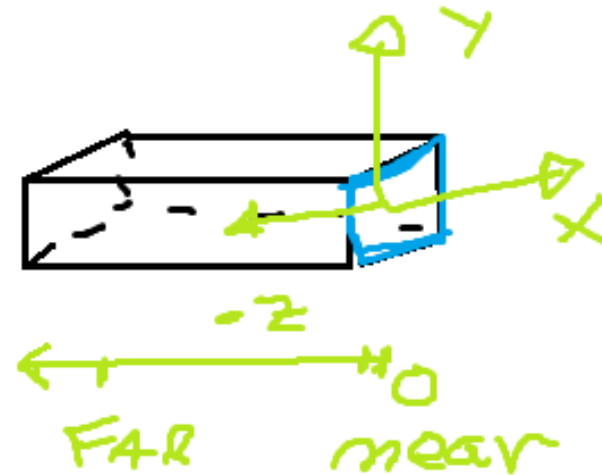
```
glm::mat4 proj = glm::ortho(-100.0f, 100.0f, -100.0f, 100.0f, 0.1f, 100.0f);
```

Left = -right

Bottom = -Top

Near = 0;

Far un valor positivo.

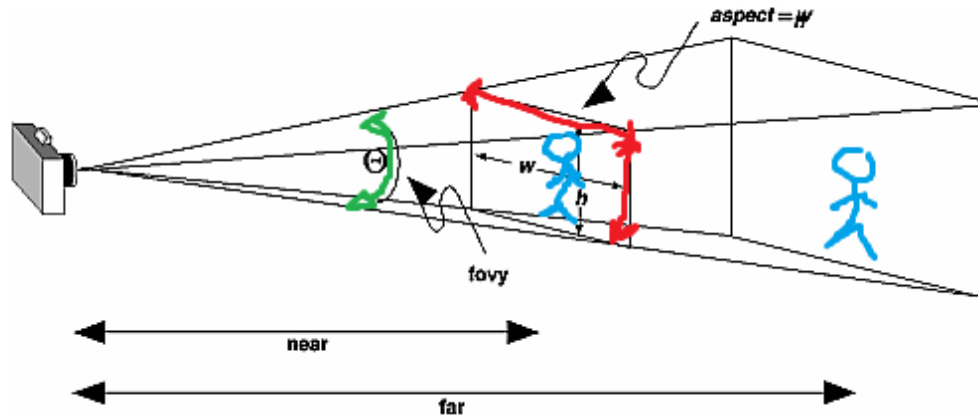


P Matriz de proyección

$$\begin{bmatrix} \frac{2}{R - L} & 0 & 0 & -\frac{(R + L)}{R - L} \\ 0 & \frac{2}{T - B} & 0 & -\frac{(T + B)}{T - B} \\ 0 & 0 & \frac{2}{F - N} & -\frac{(F + N)}{F - N} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Matriz Proyección perspectiva.

```
glm::mat4 proj = glm::perspective(45.0f, Aspect, 0.1f, 100.0f);
```



$$\begin{bmatrix} 1 / \text{Aspect} \tan(\text{fovy} / 2) & 0 & 0 & 0 \\ 0 & 1 / \tan(\text{fovy} / 2) & 0 & 0 \\ 0 & 0 & N + F / N - F & 2 \cdot F \cdot N / N - F \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Matriz Proyección perspectiva.

- Supongamos `glm::perspective` (90, 1, 1, 11);
- Calcula la matriz de proyección.
- Calcula el punto de proyección de (0, 0, -1), (0, 11, -11), (0, 4, -6);
Razona la respuesta

$$\begin{bmatrix} 1/\text{Aspt} \cdot \tan(\text{fov}/2) & 0 & 0 & 0 \\ 0 & 1/\tan(\text{fov}/2) & 0 & 0 \\ 0 & 0 & N+F/N-F & 2.F.N/N-F \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Punto	P. Coordenadas Homogéneas	Resultado	Resultado Normalizado
(0, 0, -1),			
(0, 11, -11),			
(0, 4, -6);			

Matriz Proyección perspectiva.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1.2 & -2.2 \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Punto	P. Coordenadas Homogéneas	Resultado	Resultado Normalizado
(0, 0, -1),			
(0, 11, -11),			
(0, 4, -6);			

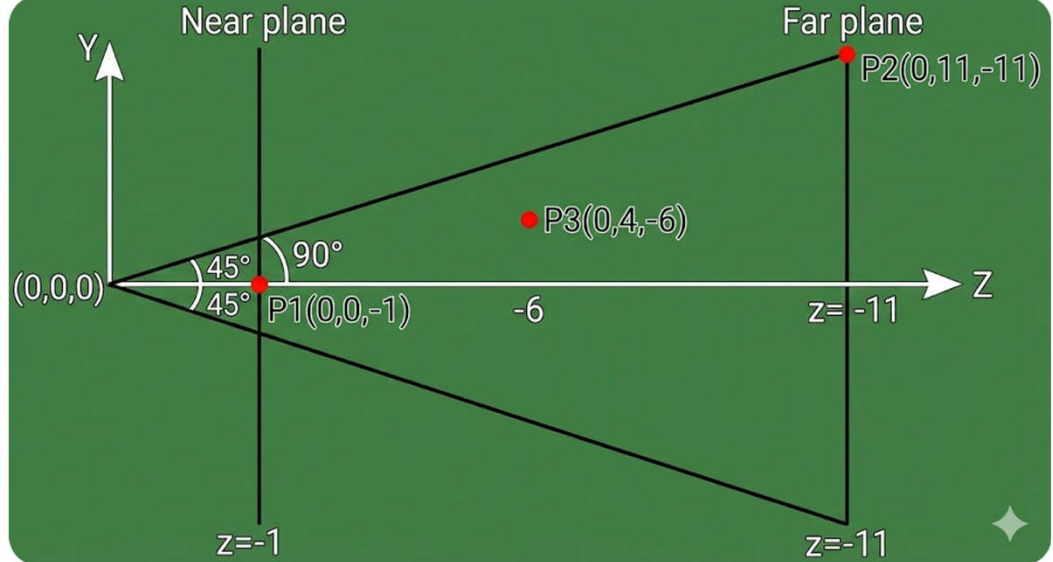
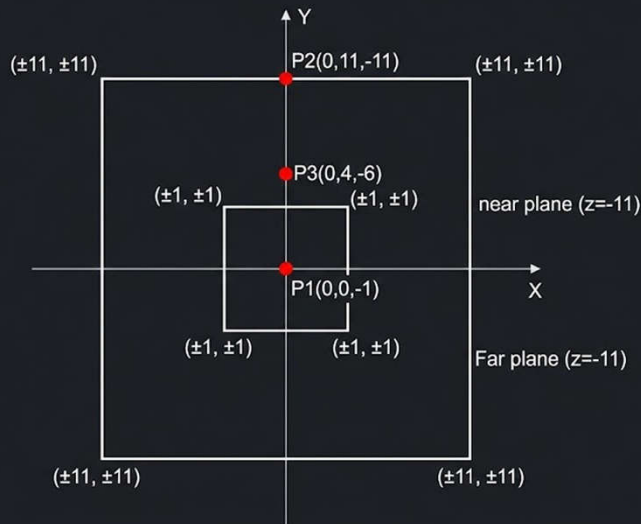
Matriz Proyección perspectiva.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1.2 & -2.2 \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Punto	P. C. Homogéneas	Resultado	Resultado Normalizado
(0, 0, -1),	(0, 0, -1, 1)	(0, 0, -1, 1)	(0, 0, -1, 1)
(0, 11, -11),	(0, 11, -11, 1)	(0, 11, 11, 11)	(0, 1, 1, 1)
(0, 4, -6);	(0, 4, -6, 1)	(0, 4, 5, 6)	0, 0.67, 0.83, 1)

gluPerspective (90, 1, 1, 11);

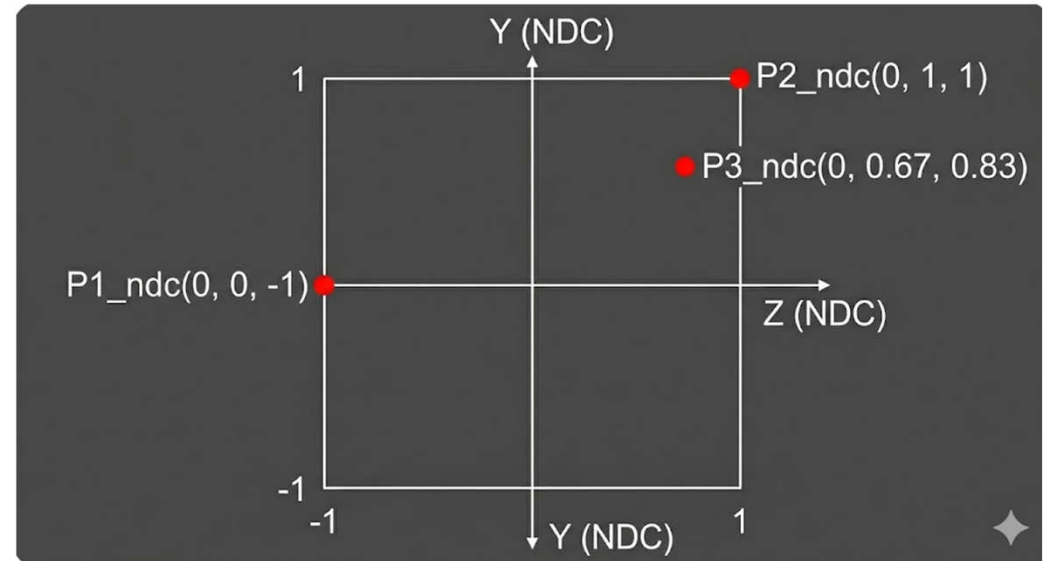
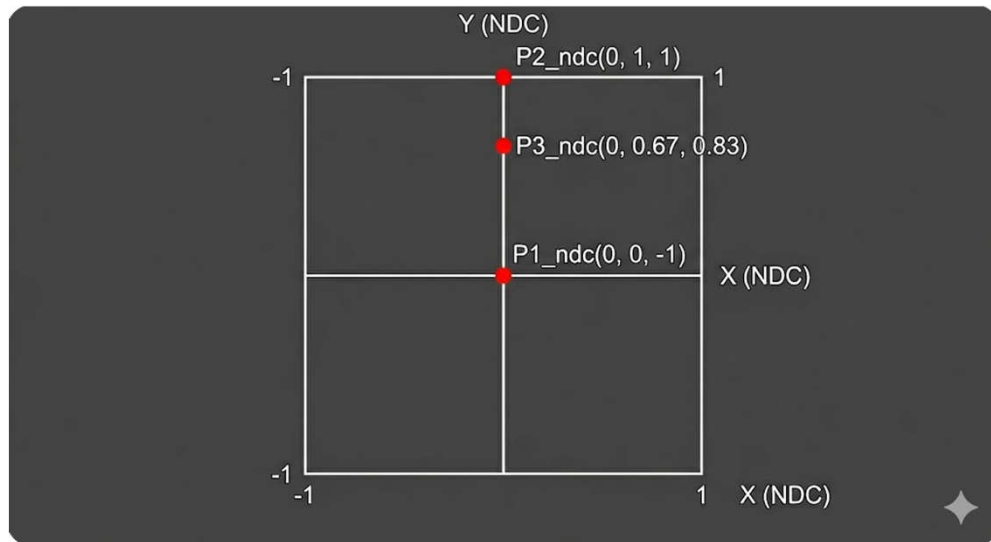
Front view (XY plane)
Aspect ratio: 1



Punto	P.C.H.	Resultado	Resultado N.
(0, 0, -1),	<i>(0, 0, -1, 1)</i>	(0, 0, -1, 1)	(0, 0, -1, 1)
(0, 11, -11),	<i>(0,11,-11,1)</i>	(0, 11, 11, 11)	(0, 1, 1, 1)
(0, 4, -6);	<i>(0, 4, -6, 1)</i>	(0, 4, 5, 6, 6)	(0, 0.67, 0.83, 1)

- **P1 (0, 0, -1):** Se ubica sobre el eje Z, justo en la línea que marca el plano cercano (*near plane*) en $z=-1$.
- **P2 (0, 11, -11):** Se ubica en la esquina superior derecha, en la intersección del plano lejano (*far plane*) en $z=-11$ y el límite superior del frustum.
- **P3 (0, 4, -6):** Se ubica en el interior del volumen de visión, a una profundidad de $z=-6$ y una altura de $y=4$.

gluPerspective (90, 1, 1, 11); Normalizada

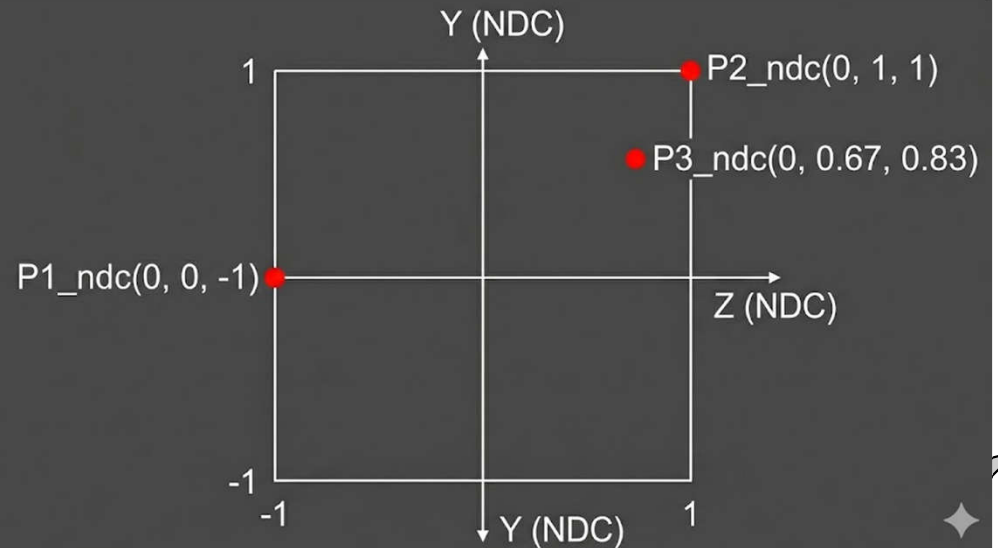
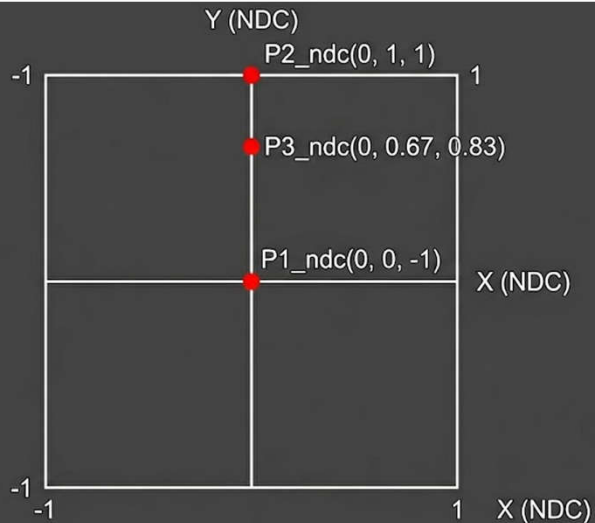
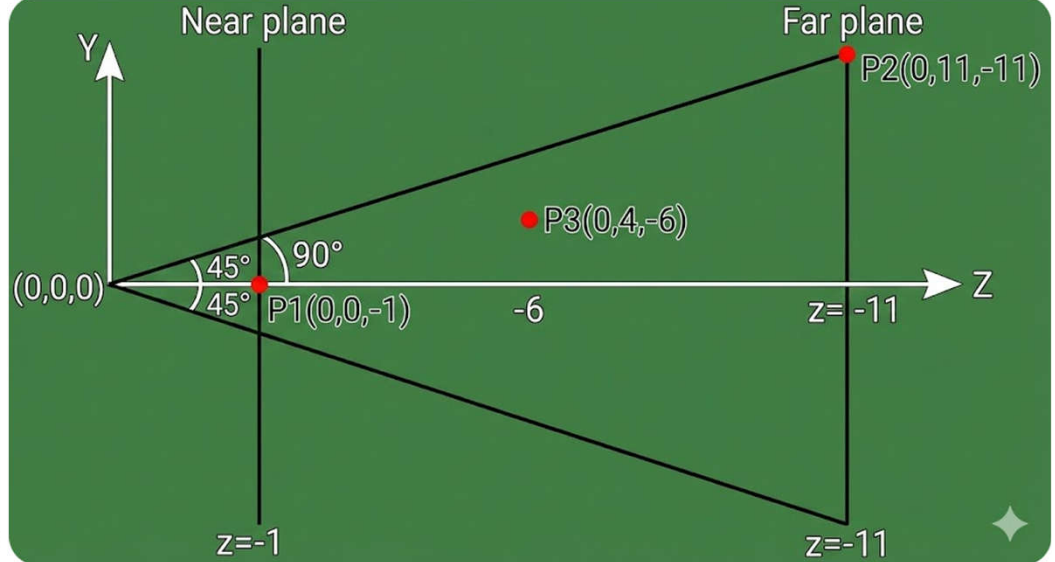
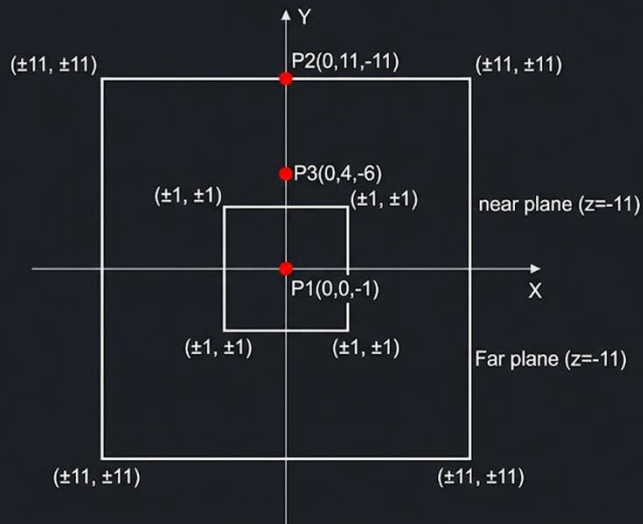


Punto	P.C.H.	Resultado	Resultado N.
(0, 0, -1),	(0, 0, -1, 1)	(0, 0, -1, 1)	<i>(0, 0, -1, 1)</i>
(0, 11, -11),	(0, 11, -11, 1)	(0, 11, 11, 11)	<i>(0, 1, 1, 1)</i>
(0, 4, -6);	(0, 4, -6, 1)	(0, 4, 5, 6, 6)	<i>(0, 0.67, 0.83, 1)</i>

- **P1 (0, 0, -1):** Se ubica sobre el eje Z, justo en la línea que marca el plano cercano (*near plane*) en $z=-1$.
- **P2 (0, 11, -11):** Se ubica en la esquina superior derecha, en la intersección del plano lejano (*far plane*) en $z=-11$ y el límite superior del frustum.
- **P3 (0, 4, -6):** Se ubica en el interior del volumen de visión, a una profundidad de $z=-6$ y una altura de $y=4$.

Comparativa Real-Normalizada

Front view (XY plane)
Aspect ratio: 1



Visión en 3D

Una imagen para cada uno de los ojos.



Visión 3D

Técnica anaglyfo.



Derecho

Izquierdo

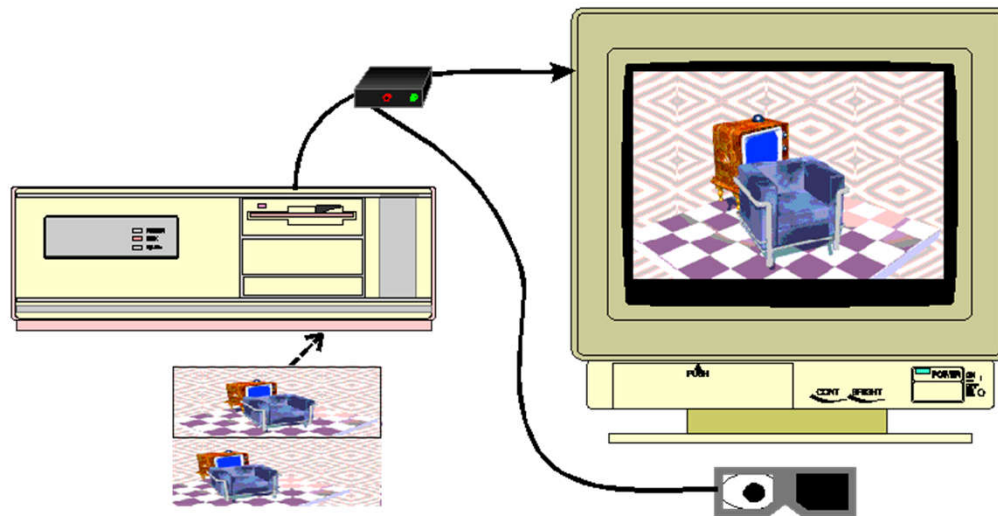
Vision 3D-Activa

Gafas activas, alternancia

Las imágenes son generadas por ojo.

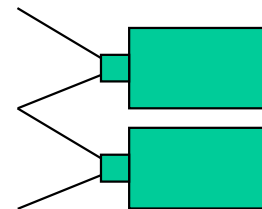
Ojo derecho, ojo izquierdo, secuencialmente.
síncronamente

Algún dispositivo filtra la información de cada ojo



Vision en 3D pasiva

- La imagen sobre la pantalla corresponde a los dos ojos con polarizaciones diferentes. La resolución se mantiene



- Se superpone una imagen sobre otra con diferente polarización.

HMD

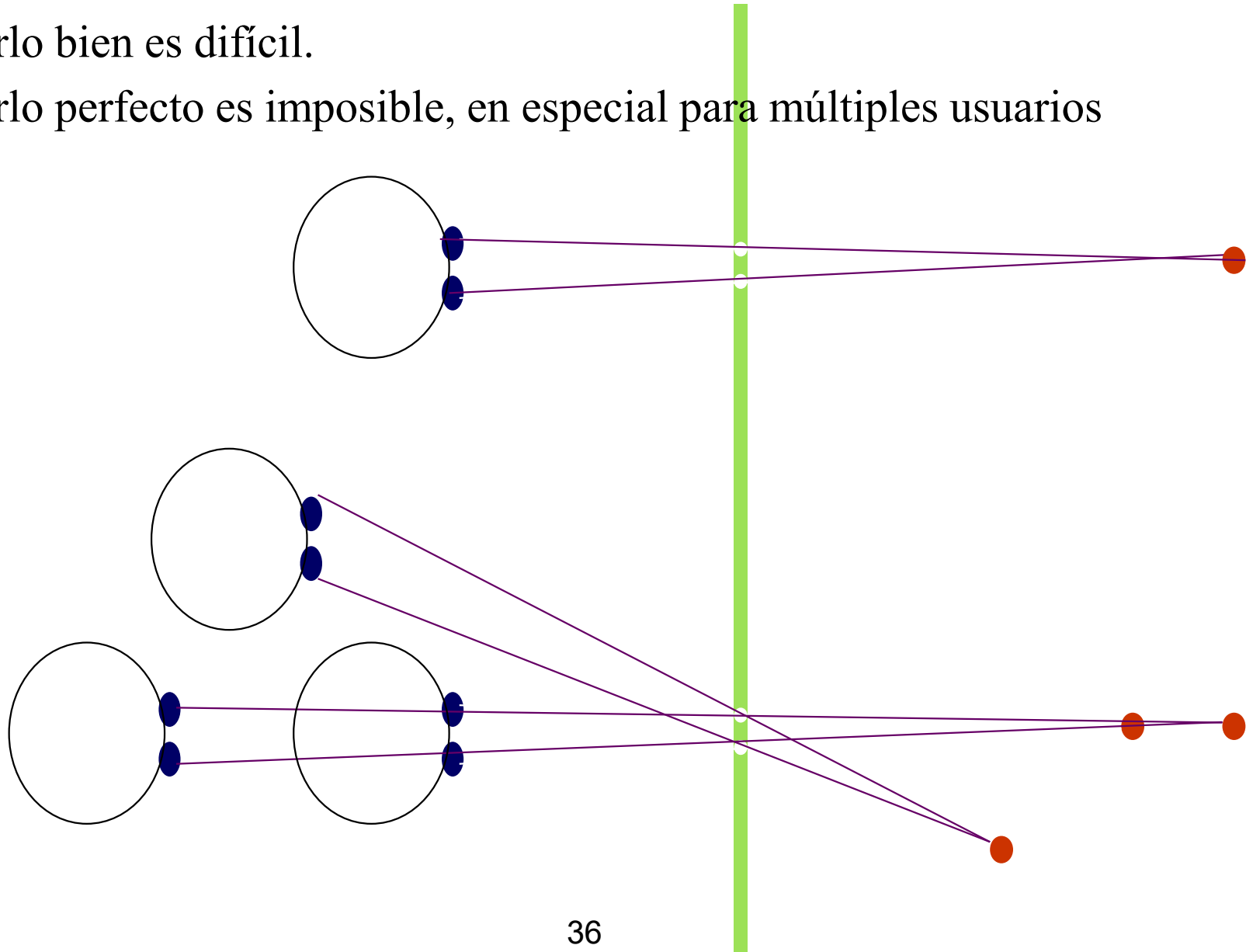


1966

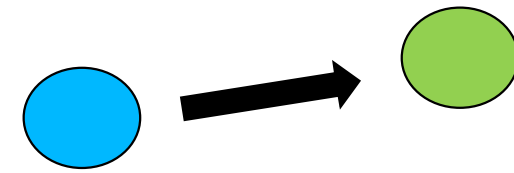


Vision 3D

- Hacer visión en 3D mal, es fácil
- Hacerlo bien es difícil.
- Hacerlo perfecto es imposible, en especial para múltiples usuarios



Cámara telescopio



```
void setupCamera() {
    glm::mat4 projection = glm::ortho(-1200.0f, 1200.0f, -1200.0f, 1200.0f, 1.0f, 3000.0f);

    glm::vec3 pos = glm::vec3(DISTANCIA_CAMARA * sin(alpha) * cos(beta), DISTANCIA_CAMARA * sin(beta), DISTANCIA_CAMARA * cos(alpha) * cos(beta));
    glm::mat4 view = glm::lookAt(pos, glm::vec3(0, 0, 0), glm::vec3(0, 1, 0));
    glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "projection"), 1, GL_FALSE, glm::value_ptr(projection));
    glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "view"), 1, GL_FALSE, glm::value_ptr(view));
}
```

```
void setupTelescope(ObjetoCeleste origen, ObjetoCeleste destino) {
    glm::mat4 projection = glm::perspective(glm::radians(45.0f), (float)SCR_WIDTH / SCR_HEIGHT, 0.1f, 3000.0f);

    glm::vec3 pOr = glm::vec3(origen.distancia * cos(origen.angulo_trans * GradosToRad), 0, origen.distancia * sin(origen.angulo_trans * GradosToRad));
    glm::vec3 pDe = glm::vec3(destino.distancia * cos(destino.angulo_trans * GradosToRad), 0, destino.distancia * sin(destino.angulo_trans * GradosToRad));
    glm::mat4 view = glm::lookAt(pOr, pDe, glm::vec3(0, 1, 0));
    glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "projection"), 1, GL_FALSE, glm::value_ptr(projection));
    glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "view"), 1, GL_FALSE, glm::value_ptr(view));
}
```

```
// --- Instancias de Planetas ---
```

```
ObjetoCeleste sol = { 0, 0, 0, 1, 0, 100 };
```

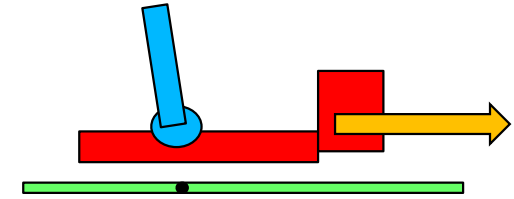
```
ObjetoCeleste mercurio = { 200, 8, 0, 2, 0, 15 };
```

```
ObjetoCeleste tierra = { 500, 5, 0, 10, 0, 80 };
```

```
ObjetoCeleste luna = { 100, 15, 0, 20, 0, 20 };
```

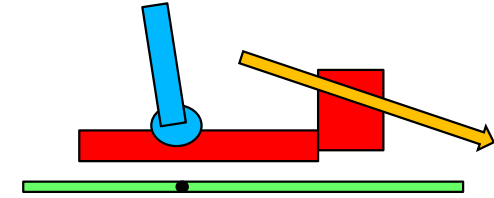
```
ObjetoCeleste marte = { 800, 3, 0, 10, 0, 45 };
```

Cámara Grúa



```
void myPrimeraPersona(objeto coche_h) {  
  
    //Projection Matriz  
    glm::mat4 projection;  
    projection = glm::mat4();  
    unsigned int projectionLoc = glGetUniformLocation(shaderProgram, "projection");  
    projection = glm::perspective(glm::radians(45.0f), (float)SCR_WIDTH / (float)SCR_HEIGHT, 1.0f, 200.0f);  
    glUniformMatrix4fv(projectionLoc, 1, GL_FALSE, glm::value_ptr(projection));  
  
    // View Matriz  
    glm::mat4 view;  
    view = glm::mat4();  
    unsigned int viewLoc = glGetUniformLocation(shaderProgram, "view");  
    view = glm::lookAt(glm::vec3(coche_h.px, coche_h.py + 6, coche_h.pz),  
        glm::vec3(coche_h.px + sin((coche_h.angulo_trans) * GradosToRadianes), 6, coche_h.pz +  
            cos((coche_h.angulo_trans) * GradosToRadianes)),  
        glm::vec3(0.0f, 1.0f, 0.0f));  
    glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));  
  
}
```

Cámara Grúa



```
void myTerceraPersona(objeto coche_h) {

// Projection Matriz
glm::mat4 projection;
projection = glm::mat4();
unsigned int projectionLoc = glGetUniformLocation(shaderProgram, "projection");//La
ponemos a uno
projection = glm::perspective(glm::radians(45.0f), (float)SCR_WIDTH / (float)SCR_HEIGHT,
1.0f, 200.0f);
glUniformMatrix4fv(projectionLoc, 1, GL_FALSE, glm::value_ptr(projection));


// View Matriz
glm::mat4 view;
view = glm::mat4();
unsigned int viewLoc = glGetUniformLocation(shaderProgram, "view");
view = glm::lookAt(glm::vec3(coche_h.px - 20 * sin((coche_h.angulo_trans) *
GradosToRadianes), coche_h.py + 10, coche_h.pz - 20 * cos((coche_h.angulo_trans) *
GradosToRadianes)),
glm::vec3(coche_h.px + sin((coche_h.angulo_trans) * GradosToRadianes), 10, coche_h.pz +
cos((coche_h.angulo_trans) * GradosToRadianes)),
glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
}
```

Temario

Tema 1: Introducción.

Introducción *100%*

Revisión histórica da computación gráfica. *Ver documentación 100%.*

Tema 2: Estándares gráficos Evolución histórica, OpenGL *40%*

Tema 3: Formas 2D e Antialiasing (final del curso) *100%*

Tema 4: Transformacións xeométricas, 2D, 3D. *100 %*

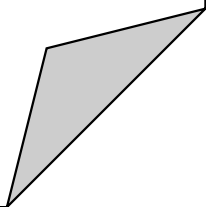
Tema 5: Proyeccións, modelo de cámara sintética. *100%*

Tema 6: Modelado e texturas *50%*

Tema 7: Color, Iluminación y sombreado

Tema 8: Determinación de superficies visibles, z-Buffer *50%*

Tema 9: Shaders GLSL



Bibliografía

- GRÁFICOS POR COMPUTADORA CON OPENGL, Hearn, Donald ; Baker, Pauline.
 - Capítulo 5.
- Redbook 8.0 o versión superior.
- <https://learnopengl.com/>